KSI Frontend Stack & UI Architecture Guide

Deep Dive into React 18, Vite 5, Tailwind CSS, Recharts & the Official iGOT Karmayogi Design System

Framework: React 18.2	**Build Tool:** Vite 5.0	**Styling:** Tailwind CSS	**Data Viz:** Recharts 2.x	**Team:** CodeVanta

SECTION 1 Modern Web Architecture: Why React 18 + Vite 5?

The KSI web portal is engineered as a high-performance Single Page Application (SPA) designed to serve administrative personnel in MoSPI. Traditional government web portals suffer from slow multi-page reloads, rigid layouts, and visual clutter. KSI replaces this with a modern component-driven frontend architecture built on **React 18** and bundled with **Vite 5**.

[PERFORMANCE] ELI20: WHY FRONTEND PERFORMANCE MATTERS

Why UI Performance Feels Like 60fps Gaming: Imagine adjusting a stat slider in an RPG menu. If the screen freezes for half a second every time you move the slider, it feels broken. In KSI, an administrator adjusts an officer's competency score from 40 to 85. **React 18's Fiber reconciler** updates the in-memory virtual DOM and recalculates the radar chart polygon at **60 frames per second** without a single server roundtrip! Meanwhile, **Vite 5** serves code using native browser ES modules, giving developers sub-50ms Hot Module Replacement (HMR) instead of waiting 30 seconds for Webpack to recompile.

SECTION 2 The Official iGOT Karmayogi Bharat Design System

To ensure seamless cognitive familiarity for civil servants, the KSI user interface strictly adheres to the official **iGOT Karmayogi Bharat** aesthetic tokens and accessibility guidelines:

Design Token	Color Name & Hex Code	Emotional Impact & Purpose	UI Component Usage
Primary Brand	Royal Blue (**#1d5ba5**)	Institutional trust, statutory authority, and clarity.	Navigation header, active tab pills, primary action buttons, radar outline.
National Accent	Karmayogi Saffron (**#f58220**)	Energy, national mission identity, and forward momentum.	Benchmark target polygon, warning badges, highlighted KPI borders.
Canvas Surface	Warm Parchment (**#fdf8f3**)	Soft, glare-free background preventing eye strain during long shifts.	Global viewport background (`bg-[#fdf8f3]`), container wrapping.
Card Surface	Clean Slate (**#ffffff / #f8fafc**)	Crisp visual separation with soft warm borders (`#e2d7cc`).	Module containers, slider cards, assessment question containers.
Status Accents	Emerald (**#10b981**) & Rose (**#e11d48**)	Instant positive/negative feedback for skill mastery vs. critical gap.	Enrolled course badges, deficit alert pills, Level 1 Bloom tags.

SECTION 3 Component Breakdown: The Three Core Views

The interface is organized into three purpose-built views accessible via the persistent top navigation bar:

1. Officer Competency Modeling & Vector Gap View (OfficerCompetencyView.jsx)

- **Dynamic Officer Selector:** Loads verified personnel profiles (e.g. *Arun Kumar, Senior Statistical Officer, FOD*) directly from SQLite via GET /api/officer/{id}.
- **Four Interactive Sliders:** Permits real-time simulation of competency improvements across Statistical Theory, Technical Tools, Data Privacy, and Managerial Decision Making on a 0-100 integer scale.
- **Live Recharts Radar Chart:** Renders two overlapping polygons-the **Current Officer Competency Vector** (Emerald shaded fill) against the **Statutory SSO Benchmark Target** (Saffron dashed perimeter). Visual gaps immediately expose operational deficits.
- **Curated iGOT Course Drawer:** Automatically displays prioritized courses matching deficit domains with instant 1-click enrollment.

2. AI Syllabus & Bloom Assessment Generator View (MCQSynthesizerView.jsx)

- **Document Upload Dropzone:** Accepts drag-and-drop of official MoSPI circular PDFs or raw text pastes (e.g. PPI Methodological Manual).
- **Async Processing Indicator:** Displays animated progress feedback while the background FastAPI engine parses text and prompts Ollama.
- **Psychometric Question Cards:** Each synthesized question displays its Bloom's Taxonomy badge (**Level 1: Recall**, **Level 2: Analysis**, or **Level 3: Application**).
- **Interactive Rationale Reveal:** When an administrator or officer selects an option, the component instantly reveals whether the choice is correct, highlighting statutory explanations derived directly from the source document.

3. Cadre Telemetry & Ministry Executive Dashboard View (CadreTelemetryView.jsx)

- **Macro KPI Stat Cards:** Displays high-level administrative metrics: *Directorates Tracked (3)*, *Mean iGOT Learning Hours (28.4 hrs)*, *Critical Domain Deficit (Digital Governance & Data Privacy)*, and *SSO Benchmark Attainment Rate (84.6%)*.
- **Division-by-Division Heatmap Table:** Granular breakdowns of the **Field Operations Division (FOD)**, **National Accounts Division (NAD)**, and **Price Statistics Division (PSD)**, highlighting specific regional training priorities.

[UI-PATTERN] UI/UX DESIGN PATTERN: OPTIMISTIC STATE UPDATES

Optimistic UI State Pattern: When an administrator clicks 'Enroll' on an iGOT course or drags a slider, the frontend updates the UI state immediately before the network request finishes. If the SQLite backend responds with an error, the state seamlessly rolls back and alerts the user with a floating toast notification. This eliminates visual stutter and gives the portal the responsiveness of a native desktop application.

SECTION 4 State Management, Radar Mathematics & API Integration

The frontend manages state cleanly using standard React hooks (useState, useEffect, useMemo) without heavyweight third-party stores like Redux. Below is the exact implementation pattern used to transform raw 4D competency vectors into polar coordinates for the Recharts Spider Radar component:

[CODE] FRONTEND: RECHARTS SPIDER RADAR COMPONENT (RadarChartComponent.jsx)

```
// RadarChartComponent.jsx - Polar Radar Rendering Engine
import React from 'react';
import { ResponsiveContainer, RadarChart, PolarGrid, PolarAngleAxis, PolarRadiusAxis, Radar, Legend, Tooltip } from 'rec
harts';

export default function RadarChartComponent({ currentScores, benchmarkScores }) {
  // Format 4D dictionary into Recharts array format
  const radarData = [
    { domain: 'Theory', current: currentScores['Theory'], benchmark: benchmarkScores['Theory'], fullMark: 100 },
    { domain: 'Tools', current: currentScores['Tools'], benchmark: benchmarkScores['Tools'], fullMark: 100 },
    { domain: 'Privacy', current: currentScores['Privacy'], benchmark: benchmarkScores['Privacy'], fullMark: 100 },
    { domain: 'Managerial', current: currentScores['Managerial'], benchmark: benchmarkScores['Managerial'], fullMark: 10
0 }
  ];

  return (
    <ResponsiveContainer width='100%' height={340}>
      <RadarChart cx='50%' cy='50%' outerRadius='80%' data={radarData}>
        <PolarGrid stroke='#e2d7cc' strokeDasharray='3 3' />
        <PolarAngleAxis dataKey='domain' stroke='#1e293b' tick={{ fill: '#1e293b', fontSize: 12, fontWeight: 'bold' }} /
>
        <PolarRadiusAxis angle={30} domain={[0, 100]} stroke='#94a3b8' />
        {/* Officer Profile: Emerald Fill */}
        <Radar name='Officer Competency' dataKey='current' stroke='#10b981' fill='#10b981' fillOpacity={0.45} />
        {/* Role Target: Saffron Perimeter */}
        <Radar name='SSO Benchmark Target' dataKey='benchmark' stroke='#f58220' fill='#f58220' fillOpacity={0.15} stroke
Dasharray='4 4' />
        <Legend wrapperStyle={{ paddingTop: 10 }} />
        <Tooltip contentStyle={{ backgroundColor: '#ffffff', borderColor: '#e2d7cc', borderRadius: 8 }} />
      </RadarChart>
    </ResponsiveContainer>
  );
}
```

SECTION 5 API Service Layer Contract (services/api.js)

Client Method	HTTP Endpoint	Payload / Query	Frontend State Action
fetchOfficer(id)	GET /api/officer/{id}	None	Populates officer profile, scores dictionary, and enrolled course IDs.
saveScores(id, scores)	POST /api/officer/{id}/scores	{ scores: {...} }	Persists modified slider values into SQLite; shows success toast.
enrollCourse(id, cId)	POST /api/officer/{id}/enroll	{ course_id: '...' }	Appends course to officer's completed list; updates course card button state.
synthesizeMCQs(file/text)	POST /api/synthesize-mcqs	Multipart Form Data	Sets loading state; triggers PyMuPDF & Ollama; renders question cards.
fetchTelemetry()	GET /api/telemetry	None	Calculates division averages and renders ministry-level KPI dashboard.